

Министерство образования и науки Российской Федерации
Санкт-Петербургский Политехнический Университет Петра Великого

—
Институт компьютерных наук и кибербезопасности

ЛАБОРАТОРНАЯ РАБОТА № 2

«ОРГАНИЗАЦИЯ ЦИФРОВОГО ВВОДА-ВЫВОДА»

по дисциплине «Аппаратные средства вычислительной техники»

Выполнил
студент гр.5151001/40001

<подпись>

Волошкевич М.А.

Преподаватель /
ассистент

<подпись>

Семенов П.О.

Санкт-Петербург
2026г.

1. Цель работы.

Изучение основ работы с цифровыми портами ввода-вывода микроконтроллера ATmega32. Получение практических навыков по обработке внешних прерываний и организации ввода-вывода с помощью механизма прерываний.

2. Схема установки

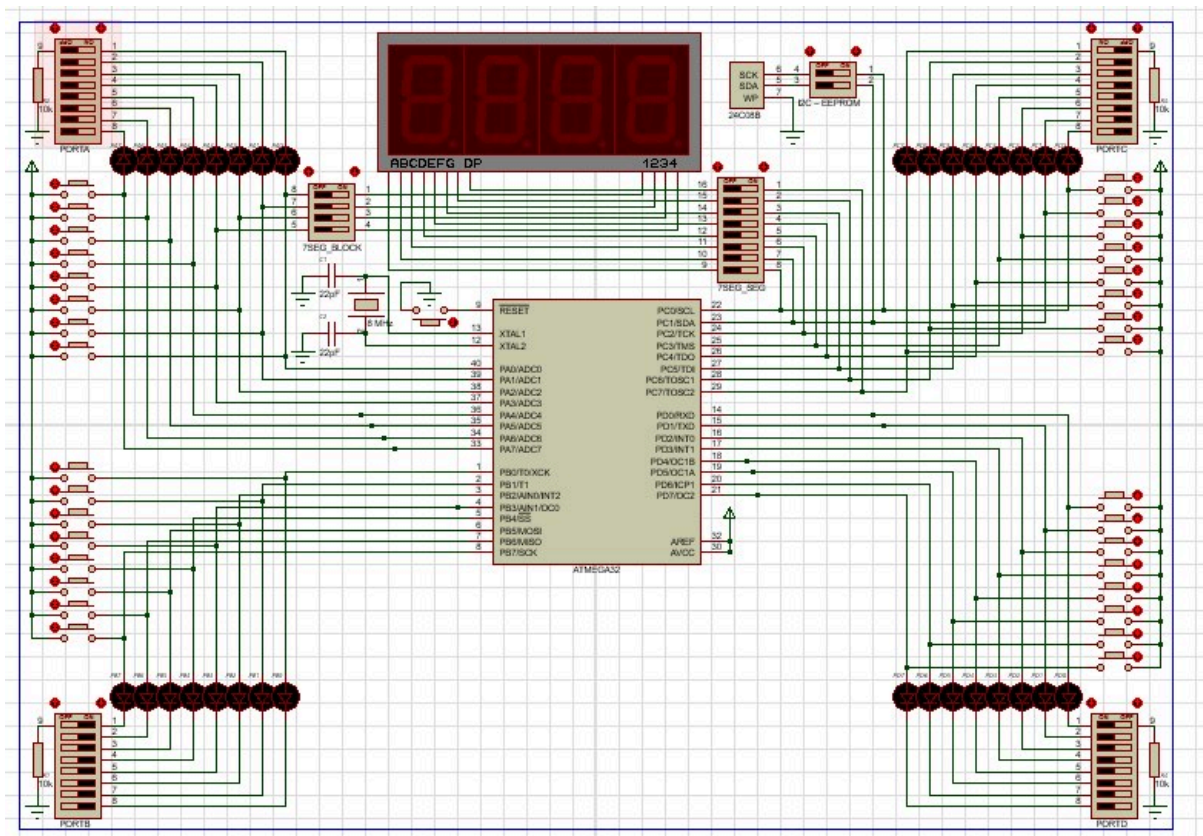


Рис. 1. Схема установки

3. Ход работы

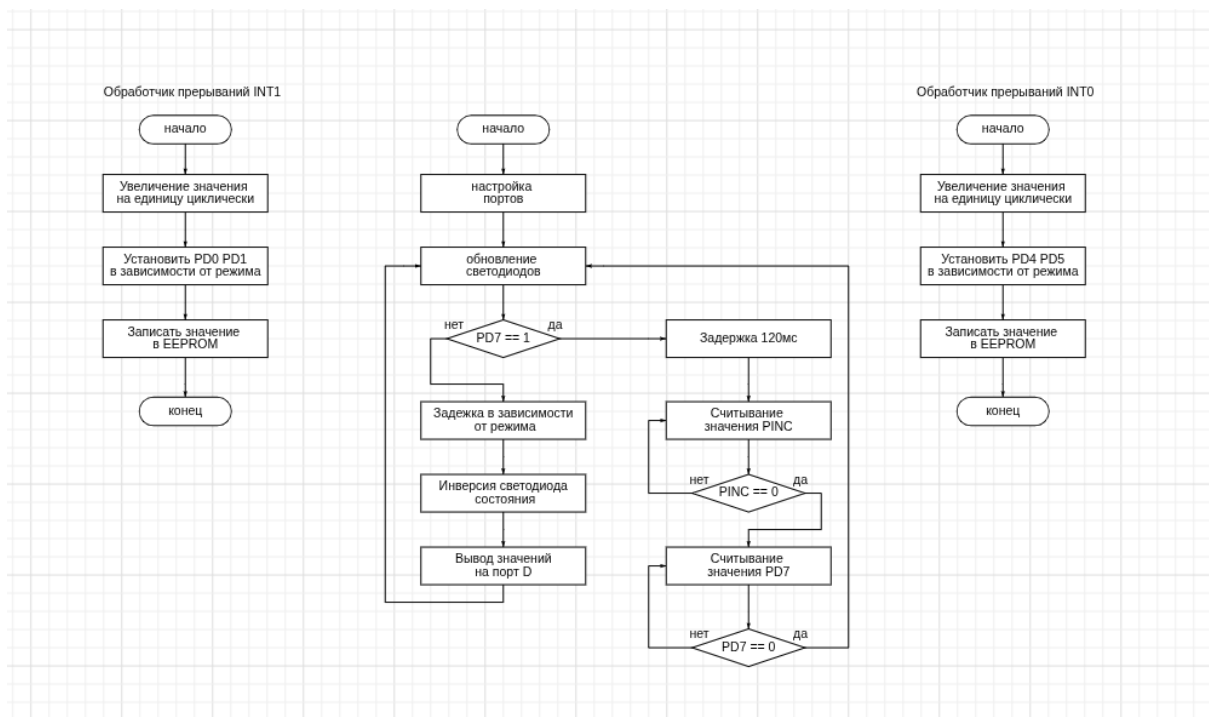


Рис. 2. Блок схема работы программы

Краткое описание работы программы:

1. Инициализация:

- Устанавливает стек
- Читает из EEPROM сохраненные частоту (FREQ_VAL) и режим (MODE_VAL)
- Настраивает биты в NUM_D: FREQ1, FREQ2, MODE1, MODE2 из EEPROM
- Настраивает порты: A, B - выходы, C - вход, D - частично выход
- Настраивает внешние прерывания INT0 и INT1 по любому изменению
- Включает глобальные прерывания

2. Основной цикл (main_loop):

- Обновляет светодиоды на портах A, B в зависимости от MODE_VAL и STATE

- Проверяет кнопку ENTER_BUTTON - вызывает input_mode
 - Вызывает delay_freq (задержка зависит от FREQ_VAL: 1сек, 0.5сек или 1.5сек)
 - Инвертирует STATE_BIT в STATE и NUM_D (переключает состояние)
 - Обновляет PORTD (выводит NUM_D)
3. Обработчики прерываний:
- INT0: увеличивает MODE_VAL (0->1->2->0), сохраняет в EEPROM, обновляет NUM_D
 - INT1: увеличивает FREQ_VAL (0->1->2->0), сохраняет в EEPROM, обновляет NUM_D
4. input_mode (кнопка ENTER):
- Читает значение с PORTC в PLUS_Y (ждет пока все кнопки отпущены)
 - PLUS_Y хранит текущее значение для режима 3
5. update_leds:
- Режим 0: светодиоды: (0xFF, 0x00)
 - Режим 1: светодиоды: (0xAA, 0x55)
 - Режим 2: светодиоды: (PLUS_Y, отрицательный PLUS_Y)
 - Если STATE=1 - меняет пары местами
6. Вспомогательные функции:
- calc_neg_y: инвертирует старший бит (получает отрицательное значение)
 - delay_freq: реализует задержку с частотой 1/0.5/1.5 секунды
 - EEPROM_read/write: чтение/запись EEPROM

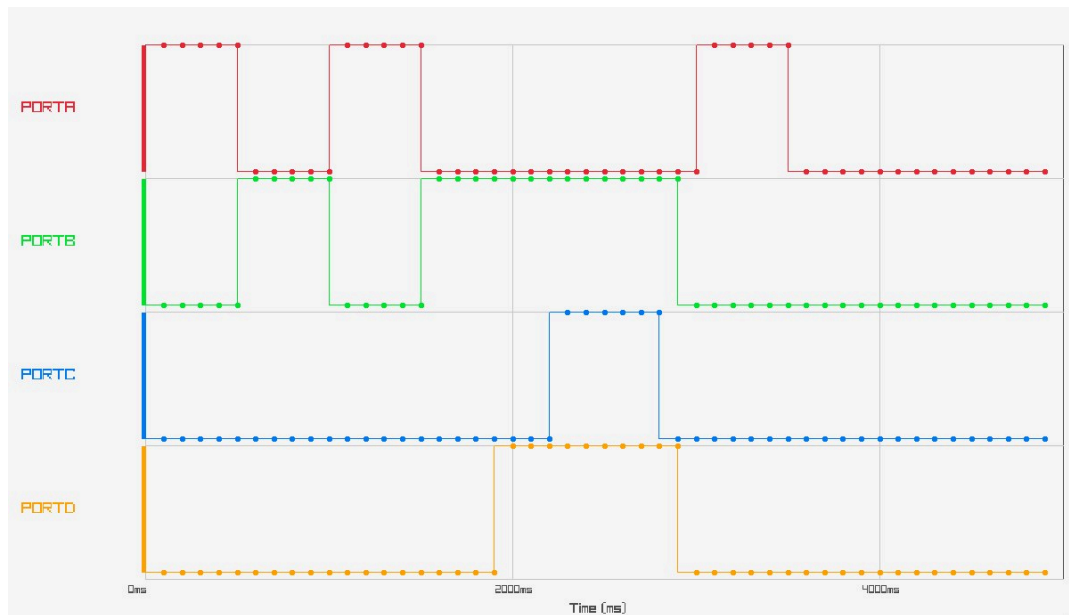


Рис. 3. Фрагмент временной диаграммы работы алгоритма

4. Ответы на вопросы

1. Какими способами можно подключить внешние устройства (светодиод, кнопку) к микроконтроллеру?
 - По логической единице
 - По логическому нулю

2. Как реализуется подсистема прерываний в микроконтроллере AVR?

Прерывания — это остановка работы основной программы для обработки события. Для каждого события есть:

- обработчик прерываний (ISR)
- вектор прерываний (адрес). Все векторы образуют таблицу векторов прерываний.

У каждого прерывания есть свой бит разрешения и есть глобальный бит разрешения прерываний SREG.

3. Как программно разрешить или запретить выполнение конкретного прерывания?

- Установить SREG в 1 для включения всех прерываний. И установить на конкретное прерывание бит разрешения в 1.

4. Какие источники прерываний есть в микроконтроллерах AVR?

- RESET
- Внешние прерывания(INT, INT1)
- Таймеры
 - Совпадение
 - Переполнение
- Интерфейсы
 - SPI
 - UART
 - I2C
- АЦП
- EEPROM готов

5. Как настраиваются внешние прерывания?

- GICR
 - Разрешает прерывания INT0, INT1
 - $INTx = 1 \rightarrow$ Разрешено
- MCUCR Задаёт условия срабатывания:
 - по уровню
 - по любому изменению
 - по фронту(0 $\rightarrow$ 1)
 - по спаду(1 $\rightarrow$ 0)

Приложения

main.asm

```
.org $000
    JMP reset
.org INT0addr
    JMP ext_int0
.org INT1addr
    JMP ext_int1

.equ FREQ1 = 0
.equ FREQ2 = 1
.equ MODE_BUTTON = 2
.equ FREQ_BUTTON = 3
.equ MODE1 = 4
.equ MODE2 = 5
.equ STATE_BIT = 6
.equ ENTER_BUTTON = 7
.equ EEPROM_ADR_FREQ = 0x10
.equ EEPROM_ADR_MODE = 0x11

.def NUM_D = R16
.def TMP = R17
.def VAL1 = R18
.def VAL2 = R19
.def STATE = R20
.def FREQ_VAL = R21
.def MODE_VAL = R22
.def TMP_CYCLE = R23
.def EEPROM_VALUE = R24
.def EEPROM_ADR = R25
.def PLUS_Y = R26
.def MINUS_Y = R27
.def NULL = R28
.def EEPROM_TMP = R29

reset:
    ; ?????????? ??????
```

```

LDI R20, HIGH(RAMEND)
OUT SPH, R20
LDI R20, LOW(RAMEND)
OUT SPL, R20

; ?????????? ??????

; ?????????? ?????????? ??????????
CLR NULL ; 0x00
LDI PLUS_Y, 0x55
CLR NUM_D
CLR STATE

LDI EEPROM_ADR, EEPROM_ADR_FREQ
CALL EEPROM_read
MOV FREQ_VAL, EEPROM_VALUE

LDI EEPROM_ADR, EEPROM_ADR_MODE
CALL EEPROM_read
MOV MODE_VAL, EEPROM_VALUE

; ?????????????????? ?????????
SBRS FREQ_VAL, 0
CBR NUM_D, (1<<FREQ1)
SBRC FREQ_VAL, 0
SBR NUM_D, (1<<FREQ1)

SBRS FREQ_VAL, 1
CBR NUM_D, (1<<FREQ2)
SBRC FREQ_VAL, 1
SBR NUM_D, (1<<FREQ2)

; ?????????????????? ?????????
SBRS MODE_VAL, 0
CBR NUM_D, (1<<MODE1)
SBRC MODE_VAL, 0
SBR NUM_D, (1<<MODE1)

SBRS MODE_VAL, 1
CBR NUM_D, (1<<MODE2)

```



```
SBRC MODE_VAL,1
SBR NUM_D,(1<<MODE2)
```

```
; ?????????? ?????? ??????-??????
```

```
SER TMP
```

```
OUT DDRA, TMP ; ?????
```

```
OUT DDRB, TMP ; ?????
```

```
CLR TMP ; 0x00
```

```
OUT DDRC, TMP ; ????
```

```
LDI TMP, 0x73 ; 0xCE
```

```
OUT DDRD, TMP ; 0,1,4,5 - ?????, 2,3,7 - ????
```

```
; ?????????? ???????????
```

```
LDI TMP, 0x0F
```

```
OUT MCUCR, TMP ; ?????????? ??????????? int0 ? int1 ?? ???????
```

```
0/1
```

```
LDI TMP, 0xC0
```

```
OUT GICR, TMP
```

```
OUT GIFR, TMP
```

```
SEI
```

```
CALL update_output
```

```
main_loop:
```

```
IN TMP, PIND
```

```
CALL update_leds
```

```
SBIC PIND, ENTER_BUTTON
```

```
CALL input_mode
```

```
CALL delay_freq
```

```
LDI TMP, (1 << STATE_BIT)
```

```
EOR STATE, TMP
```

```
EOR NUM_D, TMP
```

```
CALL update_output
```

```
RJMP main_loop
```

```

input_mode:
    CALL read_y

wait_release:
    IN TMP, PIND
    SBRS TMP, ENTER_BUTTON
    RJMP wait_release

    RET

update_leds:
    CPI MODE_VAL, 0
    BREQ mode_label1

    CPI MODE_VAL, 1
    BREQ mode_label2

    CPI MODE_VAL, 2
    BREQ mode3

mode_label1:
    LDI R29, 0xFF
    LDI R30, 0x00
    RJMP check_state

mode_label2:
    LDI R29, 0xAA
    LDI R30, 0x55
    RJMP check_state

mode3:
    MOV R29, PLUS_Y
    CALL calc_neg_y
    MOV R30, MINUS_Y

check_state:
    CPI STATE, 0
    BREQ out_leds

```

```
MOV TMP, R29
MOV R29, R30
MOV R30, TMP
```

out_leds:

```
OUT PORTA, R29
OUT PORTB, R30
```

```
RET
```

ext_int1:

```
PUSH TMP
IN TMP, SREG
PUSH TMP_CYCLE
```

```
MOV TMP_CYCLE, FREQ_VAL
CALL inc_val
MOV FREQ_VAL, TMP_CYCLE
```

```
SBRS FREQ_VAL, 0
CBR NUM_D, (1<<FREQ1)
SBRC FREQ_VAL, 0
SBR NUM_D, (1 << FREQ1)
```

```
SBRS FREQ_VAL, 1
CBR NUM_D, (1<<FREQ2)
SBRC FREQ_VAL, 1
SBR NUM_D, (1 << FREQ2)
```

```
LDI EEPROM_ADR, EEPROM_ADR_FREQ
MOV EEPROM_VALUE, FREQ_VAL
CALL EEPROM_write
```

```
POP TMP_CYCLE
OUT SREG, TMP
POP TMP
```

```
RETI
```

```

ext_int0:
    PUSH TMP
    IN TMP, SREG
    PUSH TMP_CYCLE

    MOV TMP_CYCLE, MODE_VAL
    CALL inc_val
    MOV MODE_VAL, TMP_CYCLE

    SBRS MODE_VAL, 0
    CBR NUM_D, (1 << MODE1)
    SBRC MODE_VAL, 0
    SBR NUM_D, (1 << MODE1)

    SBRS MODE_VAL, 1
    CBR NUM_D, (1 << MODE2)
    SBRC MODE_VAL, 1
    SBR NUM_D, (1 << MODE2)

    LDI EEPROM_ADR, EEPROM_ADR_MODE
    MOV EEPROM_VALUE, MODE_VAL
    CALL EEPROM_write

    POP TMP_CYCLE
    OUT SREG, TMP
    POP TMP

    RETI

EEPROM_write:
    SBIC EECR, EEWE
    RJMP EEPROM_write

    OUT EEARL, EEPROM_ADR
    CLR EEPROM_TMP
    OUT EEARH, EEPROM_TMP
    OUT EEDR, EEPROM_VALUE

    SBI EECR, EEMWE ; Master Write Enable
    SBI EECR, EEWE ; ??????? ???????

```

RET

EEPROM_read:

```
    SBIC EECR, EEWB ; ????????? ?? ?????????,
EEPROM ?????????? ???????? EEPROM, EEWB ??? ?????????? ?? ?????????????
    RJMP EEPROM_read
```

OUT EEARL, EEPROM_ADR

CLR TMP

OUT EEARH,

TMP ; ?????????????? ?????????? ?????????? ??? ?????? ?????????? ?????????? ?????? ??????????

SBI EECR, EERE ; ???

EERE ??????? ?? ?????? ?????????? ?? ?????????? ??????, ?????? ?????? ?????????????? ??????????

EEDR

IN EEPROM_VALUE, EEDR

RET

inc_val:

INC TMP_CYCLE ; ?????????? ?? 1

CPI TMP_CYCLE, 3 ; ????? >= 3

BRLO ok_inc

LDI TMP_CYCLE, 0 ; ????????? ? 0

ok_inc:

RET

update_output:

OUT PORTD, NUM_D

RET

calc_neg_y:

; ??? ?????????? ????: ?????????????? ?????????? ??? (??? ??????)

MOV MINUS_Y, PLUS_Y

LDI TMP, 0x80 ; ?????? ??? ?????????? ?????

EOR MINUS_Y, TMP ; ?????????? ?????????? ?????

RET

```

read_y: ; ?????????? ?????????
    CALL delay ; ?????????? ?????????? ?????????? ???????
    IN TMP, PINC
    MOV PLUS_Y, TMP
stop_reading: ; ?????????????? ?????????????? ??????
    IN TMP, PINC
    CP TMP, NULL ; PINC = 0?
    BRNE stop_reading ; ?????????? ?????????? PINC == 0
    RET

delay_freq:
    SBRS NUM_D,
FREQ1 ; ???? ??????? ??? ?????????????? ?????????? ?? ?????????? ?????????? ????
    RJMP delay_1
    SBRS NUM_D,
FREQ2 ; ???? ??????? ??? ?????????????? ?????????? ? ??????????
15, ???? ?? ?????????????? ?? ?????????? ? ?????????? 5
    RJMP delay_05
    RJMP delay_25
delay_1:
    LDI R29, 128
    LDI R30, 150
    LDI R31, 41
    RJMP delay_freq_loop
delay_05:
    LDI R29, 0
    LDI R30, 43
    LDI R31, 82
    RJMP delay_freq_loop
delay_25:
    LDI R29, 3
    LDI R30, 87
    LDI R31, 163
    RJMP delay_freq_loop
delay: ; ?????????? 120 ??
    LDI R31, 5
    LDI R30, 223
    LDI R29, 188

```

```
delay_freq_loop:  
    DEC R29  
    BRNE delay_freq_loop  
    DEC R30  
    BRNE delay_freq_loop  
    DEC R31  
    BRNE delay_freq_loop  
    NOP  
    NOP  
    RET
```